NAVIGATE LABS

# Capstone Project – Instructions & Guidelines

## 1. Objective

Each team must develop a **functional, multimodal, token-efficient, and secure AI-powered application** based on its assigned industry problem statement, applying concepts learned during the Generative AI Level 1 training program.

The application must solve a specific real-world problem, demonstrate meaningful Nexus AI integration, and produce reliable results.

**Priority must be given to a complete, working solution rather than an application with numerous incomplete features.** Every team member must understand the implementation.

## 2. Problem Statement Allocation & Project Scope

- Each team will be assigned one of the **30 industry-oriented problem statements** by the program coordinators.
- Teams must address the assigned problem without changing its core objective.
- All mandatory features specified in the problem statement must be implemented. Stretch features are optional.
- Teams must define a clear target user, industry scenario, and complete workflow from input to verified output.
- Consequential AI-generated results must include an appropriate human review or approval step.

### 2.1 Design Brief

Before development, teams must prepare a DESIGN.md file (1–2 pages) containing:

1. Assigned problem statement, target users, and proposed solution.
2. Core workflow, mandatory features, and application architecture.
3. Nexus AI models and their purposes.
4. At least 2 input and 2 output modalities.
5. Structured JSON output schema.
6. Human review process, safety boundaries, and 3 possible failure scenarios.
7. Test data sources and licences.

The Design Brief must be committed to GitHub and approved by the mentor before main development.

## 3. Technical Requirements & Constraints

### 3.1 Application of Course Concepts

Teams must meaningfully apply relevant concepts from the six modules:

1. Introduction to AI & Digital Fluency
2. Basics of Generative AI
3. Prompt Engineering with LLM
4. Prompt Engineering with Vision Models
5. AI with Audio & Speech Models
6. Ethical AI & Safety

The README must briefly explain where these concepts are applied.

Teams must compare **at least 2 Nexus AI models** for an important AI operation, considering response quality, token usage, and execution speed.

### 3.2 Mandatory Nexus AI Integration

- All AI models used by the application must be accessed exclusively through the **Nexus AI Platform**.
- Third-party AI model APIs and local AI models replacing Nexus capabilities are prohibited.
- Each team must use its assigned Nexus AI API key.
- AI must be a core functional component of the application.
- Any suitable programming language, framework, database, or non-AI library may be used.
- AI coding assistants may be used during development, but teams must understand the generated code.

### 3.3 Multimodal Implementation

Every application must support **at least 2 input modalities and atleast 1 output modalities** within its core workflow.

| **Modality** | **Valid Implementation** |
|---|---|
| Text Input | Text processed by a Nexus AI model |

| | |
|---|---|
| Image Input | Images, photographs, or screenshots analyzed using Nexus vision models |
| Audio Input | Recorded speech processed using Nexus speech models |
| Text Output | Model-generated reports, explanations, or structured responses |
| Audio Output | Speech generated using Nexus text-to-speech models |
| Image Output | Images generated using a supported Nexus image-generation model |

**Important:** A modality counts only when a Nexus AI model processes or generates it. Multiple text fields or document formats count as one text modality. Browser speech services, local AI models, and code-generated charts do not count.

All modalities must serve a meaningful purpose. Applications must also handle poor-quality or unsupported inputs appropriately.

**3.4 Structured Output & Human Review**

- Important AI operations must return structured JSON with appropriate validation.
- Calculations, scoring, comparisons, and business rules must be implemented in application code.
- Material factual claims and extracted fields must be traceable to their sources.
- Missing or conflicting information must be identified rather than guessed.
- Consequential AI-generated results must require human review or approval.
- Applications must handle invalid model responses and API failures.

**3.5 Evaluation Test Set**

Each team must prepare **at least 15 test cases**, including:

- Normal input scenarios.
- Missing, conflicting, or incomplete information.
- Poor-quality images or audio, where applicable.
- Unexpected or adversarial inputs.

Expected results must be prepared manually before testing. At least half of the image and audio test inputs must be realistically captured or recorded rather than generated and immediately reprocessed by AI.

Teams must report **2–3 measurable evaluation metrics** relevant to their problem statement and maintain test cases, expected results, scripts, and outcomes in the eval/ folder.

### 3.6 Token Optimization & System Prompt Design

Teams must demonstrate token efficiency during application execution and AI-assisted development.

**Runtime Optimization**

- Establish a genuine working baseline and create a Git tag named baseline.
- Measure input tokens, output tokens, and model calls using Nexus AI usage data.
- Optimize system prompts, context selection, response length, and unnecessary API calls.
- Compare baseline and final versions using the same evaluation test set.
- Demonstrate token savings without unacceptable reductions in response quality.

**System Prompt Design**

Prompts must contain clear instructions, constraints, expected output formats, and uncertainty-handling rules.

Maintain PROMPTS.md with final prompts, schemas, version history, and optimization results.

**Development Optimization**

Use AI coding assistants with focused instructions and relevant context. Document important usage decisions and available token or cost information in AI_USAGE.md.

### 3.7 Prompt Injection Security

Applications must implement safeguards against malicious instructions in user inputs, uploaded documents, images, and audio.

Required safeguards include instruction/data separation, structured output validation, code-enforced permissions, protection of credentials, and human approval for consequential actions.

Each team must conduct **at least 10 prompt injection tests**, covering all six categories:

| Category | Attack Type |
|---|---|
| A | Direct instruction override |
| B | System prompt extraction |
| C | Indirect injection through documents |
| D | Injection embedded in images |

| | |
|---|---|
| E | Injection through audio |
| F | Domain-specific unsafe requests |

For each attack, document the input, expected response, observed response, Pass/Partial/Fail result, safeguard, and screenshot.

Failures must be reported honestly. Judges may conduct additional attacks during evaluation.

**3.8 Development, Privacy & Deployment**

- Use synthetic, public, or properly licensed data only.
- Do not use real patient records, identity documents, or confidential data.
- Store API keys securely using environment variables and provide .env.example.
- All Nexus calls must go through your backend; the API key must never appear in browser code.
- Build a usable application with appropriate error handling.
- **Deploy the application and provide an accessible working link.**
- Ensure the deployed application supports live demonstration using unseen inputs.
- High-risk decisions must remain subject to human review.

**4. Responsible Use of AI Coding Assistants**

AI coding assistants are permitted, but teams are responsible for the resulting implementation.

Each team must:

1. Understand and explain its code, prompts, schemas, and architecture.
2. Maintain meaningful Git commits throughout development.
3. Document important design changes.
4. Maintain AI_USAGE.md describing tools used, AI-assisted components, and **at least 3 AI-generated mistakes and their corrections**.
5. Include up to 5 important AI development conversation links or exported transcripts with brief descriptions.

**5. Project Checkpoints**

| Checkpoint | Requirements |
|---|---|
| CP1 – Design | Approved DESIGN.md and first 5 test cases |
| CP2 – Core Application | Working core workflow, Git baseline tag, and initial token/quality measurements |
| CP3 – Submission | Complete GitHub repository, deployment, documentation, evidence, and showcase deliverables |
| Final Evaluation | Live demonstration, security testing, and individual technical evaluation |

**Checkpoint dates will be announced by the program coordinators.**

**6. Project Deliverables**

Every team must submit a **public GitHub repository** containing the following:

| Deliverable | Requirements |
|---|---|
| Source Code | Complete application, dependencies, setup instructions, and .env.example |
| README.md | Project details, assigned problem, features, architecture, Nexus models, modalities, screenshots, setup, limitations, and all required links |
| DESIGN.md | Approved design brief and changes |
| PROMPTS.md | Final prompts, schemas, versions, and optimization evidence |
| AI_USAGE.md | AI coding assistant usage and development records |
| eval/ | Minimum 15 test cases, expected outputs, scripts, and results |
| evidence/ | Screenshots, Nexus token usage, model comparison, security testing, and human review evidence |
| slides/ | Final presentation of maximum 7 slides |
| project-writeup.pdf | Strictly one-page project write-up |
| Demo Video | Maximum 3-minute video or accessible link |
| LinkedIn Post | Public project showcase link |

### 6.1 Evidence Requirements

The evidence/ folder must contain screenshots and a proofs.txt file covering:

- Nexus model comparison.
- Baseline versus optimized token usage and quality.
- Input and output modality demonstrations.
- Prompt injection tests and safeguards.
- Human review workflow.
- Relevant AI development conversation records.

All evidence must be genuine, verifiable, and free of sensitive information.

### 6.2 One-Page Project Write-Up

Each team must prepare a **strictly one-page PDF** containing:

- Project title, domain, and team details.
- Industry problem and proposed solution.
- Key features and Nexus AI technologies.
- Brief application workflow and industry impact.
- **Working deployed application link.**
- **Public GitHub repository link.**

Upload it as project-writeup.pdf.

### 6.3 Project Demonstration Video

Create a **maximum 3-minute video** explaining the problem, solution, important features, and Nexus AI integration.

The video must include a **live demonstration of the working application** showing its core functionality. All team members should contribute.

Upload the video to GitHub or include an accessible video link in the README.

### 6.4 Mandatory LinkedIn Project Showcase

Each team must publish a LinkedIn post that:

1. Mentions the **Navigate Labs – Generative AI Level 1 Training Program at Chandigarh University**.
2. Highlights important concepts and skills learned.
3. Introduces the assigned problem and developed solution.
4. Explains key features, AI integration, and industry relevance.
5. Includes the **one-page project write-up and 3-minute demonstration video**.
6. Provides the **deployed application and GitHub repository links**.
7. **Tags Navigate Labs, Chandigarh University, and AIT/CSE.**

The public LinkedIn post URL must be included in the GitHub README.

### 6.5 Final Submission

Teams must submit the **public GitHub repository link** containing all required deliverables, evidence, and accessible project links.

## 7. Presentation Guidelines – Maximum 7 Slides

| Slide | Content |
|---|---|
| 1 | Project Title, Domain & Team |
| 2 | Assigned Problem, Target User & Solution |
| 3 | Architecture, AI Workflow & Human Review |
| 4 | Nexus Models & Multimodal Implementation |
| 5 | System Prompt Design & Token Optimization |
| 6 | Prompt Injection Security & Testing |
| 7 | Evaluation Results, Live Demo & Conclusion |

**8. Final Evaluation Format**

Each team will have **20 minutes** for evaluation.

| Activity | Duration | Description |
|---|---|---|
| Live Demonstration | 6 minutes | Demonstrate the deployed application using an unseen input |
| Security Testing | 4 minutes | Judges perform 2–3 prompt injection attacks |
| Individual Technical Viva | 8 minutes | Every member explains code, prompts, architecture, and implementation decisions |
| Questions & Handover | 2 minutes | Final questions |
| **Total** | **20 minutes** | |

Judges may request a small technical modification or ask students to explain how it would be implemented.

**9. Evaluation Criteria – 100 Marks**

| S.No. | Evaluation Metric | Marks |
|---|---|---|
| 1 | **Problem Understanding & Innovation** – Industry relevance, originality, feasibility, and solution design | 10 |
| 2 | **Application of Course Concepts** – Prompt engineering, vision, speech, structured outputs, model comparison, and responsible AI | 15 |
| 3 | **Nexus AI Integration & Multimodal Implementation** – Correct integration and at least 2 input and 2 output modalities | 20 |
| 4 | **Token Efficiency & System Prompt Optimization** – Baseline comparison, measurable savings, prompt quality, and supporting evidence | 20 |
| 5 | **Prompt Injection Security & Robustness** – Safeguards, adversarial testing, documented evidence, and live security performance | 15 |
| 6 | **Application Functionality & Reliability** – Working features, deployment, evaluation, error handling, and human review | 10 |
| 7 | **Presentation, Documentation & Project Showcase** – GitHub, write-up, video, LinkedIn, presentation, and technical understanding | 10 |
| | **Total** | **100** |

### 9.1 Individual Technical Understanding

Every team member must understand the application and participate in the technical viva. Individual marks may vary based on demonstrated understanding.

### 9.2 Academic Integrity & Penalties

- **Prohibited AI models:** Nexus integration marks may be forfeited; serious violations may lead to disqualification.
- **Copied projects or fabricated evidence:** Disqualification.
- **Exposed credentials or confidential data:** Mark deductions and mandatory corrective action.
- **Missing deliverables:** Deductions under the relevant evaluation criteria.

## 10. Important Instructions

1. Work only on the **assigned problem statement** and complete its mandatory features.
2. Use **Nexus AI exclusively** for application AI models.
3. Implement **at least 2 input and 2 output modalities**.
4. Obtain approval for DESIGN.md before main development.
5. Compare at least **2 Nexus AI models** and maintain **15 evaluation test cases**.
6. Demonstrate token optimization using a genuine baseline and verifiable Nexus usage data.
7. Conduct **10 prompt injection tests across all 6 categories**.
8. Deploy the working application and maintain a complete public GitHub repository.
9. Submit a **strictly one-page write-up**, **maximum 3-minute video**, and **LinkedIn showcase**.
10. Tag **Navigate Labs, Chandigarh University, and AIT/CSE** in the LinkedIn post.
11. Every team member must understand and demonstrate the implementation.
12. Copied projects, fabricated evidence, exposed credentials, and misuse of sensitive data are prohibited.

---

**Final Evaluation:** Teams will be assessed on their ability to develop and deploy a practical, multimodal, token-efficient, and secure AI application using Nexus AI, supported by reliable functionality, measurable results, documented security testing, individual technical understanding, and a professional project showcase.